Universidad de San Buenaventura

Programa de Ingeniería en Sistemas

AutoMarket — Componente backend

Sprint 3 — Consolidación, MySQL, HTTPS, auditoría y fotos

Materia	Laboratorio de Software III
Docente	Iván M. Cabezas T.
Integrantes	Mejía Chavarriaga – Samuel Rodriguez Castro – Juan Felipe Vinasco López – Violeta
Periodo	2026-2
Fecha	19/10/2026

1. Introducción y objetivo

El **Sprint 3** cierra el proyecto en cuatro frentes. Primero, **consolida el sistema de Accounting** descentralizado añadiendo **integridad verificable** a la traza de auditoría. Segundo, incorpora la **carga de fotos** en las publicaciones, con un **máximo de 22 imágenes por vehículo**. Tercero, atiende el cambio de última hora del docente: **migrar toda la persistencia a un motor de base de datos MySQL**. Cuarto, **publicar la aplicación en un servidor con HTTPS**.

Objetivos específicos:

- Garantizar que la traza de auditoría sea **trazable e íntegra**, con detección de alteraciones.
- Permitir que el vendedor adjunte hasta **22 fotos** por publicación, validando su contenido real.
- Almacenar usuarios, publicaciones, sesiones, cierres de sesión, auditoría y fotos en **MySQL**.
- Servir las imágenes desde la base de datos y mantener la Separación Estructural.
- Desplegar la aplicación con **HTTPS** en un servidor gratuito y una base de datos gestionada.

2. Alcance y trazabilidad

Historia / Requisito	Capacidad del Sprint 3
HU-VIS-01	El catálogo público muestra las fotografías de cada vehículo junto a los datos de contacto
HU-VEN-01	El vendedor adjunta fotos (máx. 22) al publicar un vehículo
HU-ADM-01	El administrador ve el número de fotos de cada publicación pendiente y accede a la auditoría
RF-01	Accounting con cadena de integridad SHA-256 y vista de verificación
Cambio de última hora	Persistencia en MySQL para toda la información y despliegue con HTTPS

La carga de fotos es una **funcionalidad pedida por el cliente** dentro del alcance de las publicaciones; no altera las historias de usuario, sino que amplía el dato publicado. El cambio de MySQL y HTTPS tampoco altera las HUs: sustituye la infraestructura de persistencia y el canal de publicación.

3. Arquitectura y diseño

Cambios relevantes sobre el Sprint 2:

- `internal/domain`: `Publication` incorpora `Photos []string`; `AuditLog` incorpora `PrevHash` y `Hash`; se definen `MaxPhotosPerPublication = 22` y `Photo`.
- `internal/storage/mysql`: **nuevo adaptador de producción** que implementa el puerto `Repository` con `database/sql` y `github.com/go-sql-driver/mysql`.
- `internal/storage`: la implementación JSON se conserva únicamente como **doble de pruebas**; la aplicación nunca la usa en producción.
- `internal/service`: la creación de publicación valida el máximo de fotos y se añade `AuditIntegrity` (solo Administrador); las fotos se guardan y recuperan a través del puerto, sin que el servicio conozca MySQL.

- `web/handlers`: se atiende el formulario `multipart/form-data`, se validan las imágenes, se sirve `/media/:id` y se expone la vista `/admin/audit`.
- `web/templates`: se añade `audit.html` y galerías de miniaturas en el catálogo y el panel.
- `cmd/gencert`: genera un certificado TLS autofirmado con la biblioteca estándar.
- `render.yaml` y `docker-compose.yml`: despliegue con HTTPS y MySQL.

3.1 Esquema de la base de datos

Tabla	Contenido
`users`	cuentas de usuario (id, usuario, contraseña, rol)
`publications`	publicaciones de vehículos y las URLs de sus fotos
`sessions`	sesiones activas con su fecha de expiración
`logouts`	historial de cierres de sesión
`audit`	traza de auditoría encadenada por hash
`audit_state`	último hash de la cadena (detecta borrados al final)
`photos`	contenido binario de las imágenes (`LONGBLOB`) y su tipo MIME

El esquema se crea de forma **idempotente** al arrancar (`CREATE TABLE IF NOT EXISTS`), de modo que el despliegue no requiere migraciones manuales.

4. Separación Estructural (SS)

- **Lógica vs. E/S:** el servicio nunca manipula archivos de imagen ni HTTP; recibe el contenido ya validado y devuelve identificadores. La lectura del formulario, la validación del contenido real y la respuesta HTTP ocurren en `web/handlers`.
- **Persistencia aislada:** todo el SQL vive en `internal/storage/mysql`. El servicio solo conoce el puerto `Repository`; no importa `database/sql` ni el driver.
- **`main` conciso:** solo lee la configuración (`DATABASE_DSN`, `PORT`, TLS), construye el almacenamiento, siembra el administrador y arranca Gin.
- **Dominio puro:** sin dependencias de I/O ni de la base de datos.

5. Marco de seguridad AAA (RF-01)

Pilar	Implementación en el Sprint 3
Authentication	Cookie de sesión validada por el middleware y por el servicio; las sesiones viven en MySQL.
Authorization	Doble verificación de rol; la verificación de integridad de la auditoría es exclusiva del Administrador. Los accesos denegados a listados sensibles también se auditan.
Accounting	Cada entrada de la tabla `audit` se **encadena** con el hash de la anterior, aportando integridad además de trazabilidad.

5.1 Integridad de la traza

Cada entrada incluye su hash y el de la entrada previa:

$$hash_n = SHA\text{-}256(hash_{n-1} \parallel id \parallel fecha \parallel acción \parallel usuario \parallel rol \parallel éxito \parallel detalles)$$

- La primera entrada de la cadena usa `prev_hash` vacío.
- `SaveAuditLog` calcula el hash dentro de una transacción y bloquea `audit_state` para serializar escrituras concurrentes.
- `VerifyAuditIntegrity` recalcula toda la cadena y compara. Si algún registro se altera, reordena o elimina, la cadena se rompe y la vista lo reporta como **alterada**. El último hash de `audit_state` detecta además el borrado de las últimas entradas.
- La vista `/admin/audit` muestra la traza y el estado **íntegra / alterada**.

6. Funcionalidad de fotos

- **Cantidad:** hasta **22** fotos por publicación. El límite se valida en el adaptador web (antes de guardar) y en el servicio (regla de negocio `ErrTooManyPhotos`).
- **Tipos permitidos:** JPEG, PNG, WEBP y GIF. El tipo se determina por el **contenido real** del archivo (`http.DetectContentType`), no por su nombre o extensión.
- **Tamaño:** máximo 5 MB por imagen; el formulario se acota con `http.MaxBytesReader`.
- **Almacenamiento:** el binario se guarda en la tabla `photos` de MySQL (`LONGBLOB`) con un identificador aleatorio. La publicación guarda las URLs públicas (`/media/<id>`), no el binario.
- **Servicio:** `GET /media/:id` recupera la imagen de la base de datos y la entrega con su `Content-Type`.
- **Consistencia:** si la publicación no llega a crearse, las fotos recién insertadas se eliminan. Al eliminar una publicación también se eliminan sus fotos.

7. Pruebas y evidencia

```
$ go test ./...
?       automarket/cmd/api  [no test files]
?       automarket/cmd/gencert  [no test files]
?       automarket/internal/domain  [no test files]
ok      automarket/internal/service
ok      automarket/internal/storage
ok      automarket/internal/storage/mysql
ok      automarket/web/handlers

$ go vet ./...
(sin observaciones)

$ gofmt -l .
(sin archivos por formatear)
```

Pruebas añadidas en este sprint:

- **Servicio:** el límite de 22 fotos se acepta en el borde y se rechaza al superarlo; las fotos quedan asociadas a la publicación; `AuditIntegrity` exige rol Administrador y detecta una traza alterada.
- **Storage:** la cadena de hashes es válida al crearse y tras recargar desde disco, y se detecta cuando un registro es modificado manualmente.

- **MySQL:** prueba de integración que cubre usuarios, publicaciones, sesiones, auditoría y fotos, incluida la detección de una alteración deliberada. Se ejecutó contra el contenedor MySQL (`AUTOMARKET_TEST_DSN`) con resultado **PASS**; también se omite si no se define la variable.
- **Integración HTTP:** publicación con fotos (que luego se sirven con `Content-Type` `image/png`), rechazo al exceder 22 imágenes y consulta de la vista de auditoría.
- **Docker Compose:** MySQL y aplicación quedan *healthy*; el administrador se crea en la base de datos y el flujo completo (registro, publicación con foto, aprobación, catálogo y auditoría) se verificó contra los contenedores.

8. Restricciones verificadas

- **Dependencias externas autorizadas:** `github.com/gin-gonic/gin v1.11.0` y `github.com/go-sql-driver/mysql v1.8.1`. El cambio de última hora del docente autoriza MySQL; el resto del proyecto sigue usando la biblioteca estándar (imágenes, SHA-256, certificados TLS, multipart).
- **Protocolo:** HTTPS en el despliegue (TLS terminado por Render) y HTTPS local opcional con certificado autofirmado generado con `crypto/x509`.
- **Persistencia:** MySQL/MariaDB; el JSON solo se conserva como doble de pruebas.
- **Separación Estructural:** la capa web no ejecuta SQL y el servicio no conoce MySQL.

9. Dificultades encontradas y soluciones

- **Integridad sin dependencias.** Se descartó cualquier librería criptográfica externa; se implementó la cadena con `crypto/sha256` y una serialización canónica de los campos.
- **Precisión de la fecha en el hash.** MySQL almacena microsegundos; la marca de tiempo se normaliza a UTC y se trunca a microsegundos antes de calcular el hash para que la verificación sea estable al releer la base de datos.
- **Escrituras concurrentes de auditoría.** Se añadió `audit_state` con un bloqueo `SELECT ... FOR UPDATE` dentro de la transacción, evitando cadenas paralelas.
- **Fotos en la base de datos.** Cada imagen se guarda como `LONGBLOB` y se sirve por separado, evitando cargar todas las fotos de una publicación en memoria.
- **Despliegue gratuito.** Render no incluye MySQL gratuito; se documentó el uso de TiDB Cloud Serverless o Aiven y se dejó el DSN en una variable de entorno.

10. Conclusiones

El Sprint 3 deja el sistema consolidado: el Accounting no solo registra la traza, sino que **demuestra su integridad**; las publicaciones pueden incluir hasta 22 fotos validadas y persistidas; y toda la información vive en **MySQL**. La arquitectura se mantiene limpia (I/O, lógica y persistencia separadas), el despliegue queda preparado para **HTTPS** en Render y el conjunto cumple el alcance de las HUs y RF-01 con las dependencias autorizadas para el cambio de última hora.

11. Declaración de uso responsable de IA

Se emplearon herramientas de asistencia con inteligencia artificial como apoyo en la implementación de la cadena de integridad, la gestión de fotos, la migración a MySQL, la configuración de HTTPS y el despliegue, las pruebas y la redacción. Todas las propuestas fueron revisadas, ejecutadas y verificadas

por el equipo (`go test`, `go vet`, `gofmt` y pruebas end-to-end sobre Docker), y las decisiones de diseño fueron discutidas y asumidas por sus integrantes. El equipo se responsabiliza del contenido final y del uso ético de estas herramientas.